

บทที่ 3

จาวาเซิร์ฟเวอร์เพจ

จาวาเซิร์ฟเวอร์เพจ (Java Server Pages : JSP) เป็นเทคโนโลยีที่ใช้ “สคริปต์” ในการพัฒนาโปรแกรมประยุกต์เว็บเพจ (Web Application) เพื่อทำงานทางฝั่งเซิร์ฟเวอร์ (Server-side-script) และส่งผลกลับมายังเว็บเบราว์เซอร์เป็นภาษา HTML เหมือนกับเทคโนโลยีอื่นๆ เช่น ASP , PHP เป็นต้น

การเขียนสคริปต์ JSP จะใช้ภาษาจาวาเพื่อเพิ่มประสิทธิภาพให้หน้าเว็บเพจมีความยืดหยุ่นมากขึ้น โครงสร้างของ JSP จะเป็นลักษณะของแท็ก (tag) ชนิดพิเศษที่แทรกเข้าไปในเอกสาร HTML และเปลี่ยนนามสกุลเป็น .jsp แทนที่จะเป็น .html โดยแท็กเหล่านี้เว็บเบราว์เซอร์ (web browser) จะไม่สามารถตีความได้ จะต้องนำไปประมวลผลก่อนที่เว็บเซิร์ฟเวอร์ก่อน แล้วนำผลลัพธ์ทั้งหมดส่งกลับมายังเว็บเบราว์เซอร์ ในลักษณะของเอกสาร HTML ซึ่งเว็บเบราว์เซอร์สามารถตีความและนำมาแสดงผลได้

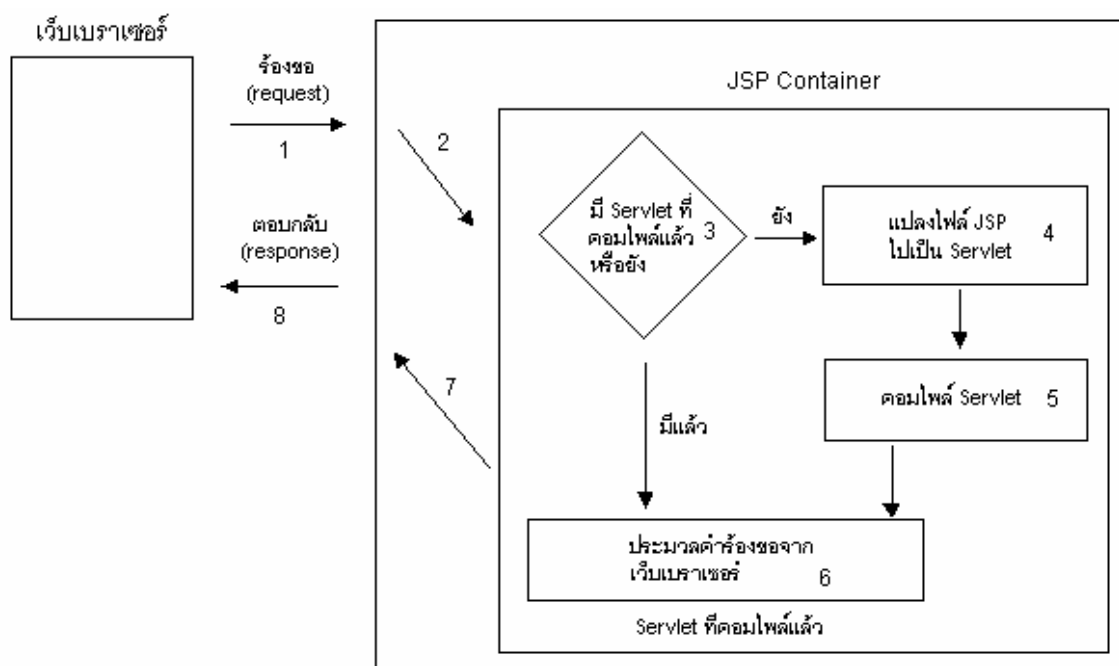
หัวใจของ JSP คือภาษาจาวา ซึ่งเป็นภาษาที่มีหัวใจหลักอยู่ที่ออบเจกต์ ซึ่งช่วยให้ง่ายต่อการพัฒนาในโปรเจกต์ใหญ่ๆ ตลอดจนสามารถนำเอาส่วนประกอบหรือคอมโพเนนต์ต่างๆ (component) กลับมาใช้ได้อีก ซึ่งเป็นประโยชน์อย่างยิ่งโดยเฉพาะในการพัฒนาโปรแกรมขนาดใหญ่

3.1 โครงสร้างและขั้นตอนการทำงานของ JSP

สิ่งที่มีความสำคัญในการทำงานของ JSP ได้แก่ JSP Container (หรือที่เรียกอีกอย่างหนึ่งว่า JSP Engine) ซึ่งเป็นส่วนประกอบสำคัญที่อยู่ในเว็บเซิร์ฟเวอร์ เพราะทำหน้าที่ควบคุมและประมวลผลไฟล์ JSP ที่มีการร้องขอ (request) เข้ามา และตอบสนอง (response) คำขอร้องขอนั้นกลับไปยังไคลเอนต์

ขั้นตอนการประมวลผลไฟล์ JSP ทั้งหมด แบ่งเป็น 8 ขั้นตอน ดังนี้

1. ฟังไคลเอนต์ส่งคำร้องขอเอกสาร JSP ไปที่เว็บเซิร์ฟเวอร์
2. เว็บเซิร์ฟเวอร์ตรวจสอบคำร้องขอ พบว่าเป็นไฟล์ JSP จึงส่งต่อไปให้แก่ JSP Container
3. JSP Container ตรวจสอบว่าไฟล์ JSP ที่ร้องขอมา เคยแปลงเป็น Servlet หรือเปล่า และคอมไพล์เป็นไฟล์ .class อยู่หรือเปล่า ถ้ายังไม่มี ก็จะกระโดดข้ามไปทำงานตามขั้นตอนข้อ 4 ต่อ แต่ถ้ามีอยู่แล้ว ก็จะตรวจสอบต่ออีกว่า หลังจากแปลงไฟล์ JSP เป็น Servlet และ คอมไพล์เป็นไฟล์ .class ครึ่งล่าสุดแล้ว ไฟล์ JSP นั้นจะมีการเปลี่ยนแปลงแก้ไขหรือเปล่า ถ้ามีการแก้ไข ก็จะกระโดดไปทำงานขั้นตอนข้อ 4 ต่อเช่นกัน แต่ถ้าไม่มีการแก้ไข แสดงว่า ไฟล์ JSP นั้นยังคงเดิมไม่เปลี่ยนแปลงจึงข้ามไปทำงานขั้นตอนข้อ 6 ได้เลย
4. JSP Container แปลงไฟล์ JSP เป็น Servlet
5. JSP Container คอมไพล์ไฟล์ Java Servlet เป็นไฟล์ .class
6. JSP Container ประมวลผลตามคำร้องขอนั้น
7. JSP Container ส่งผลลัพธ์ที่ได้จากการประมวลผล ให้แก่เว็บเซิร์ฟเวอร์
8. เว็บเซิร์ฟเวอร์ส่งผลลัพธ์นั้นไปยังไคลเอนต์หรือเว็บเบราว์เซอร์อีกทอดหนึ่ง



รูปที่ 3-1 รูปแสดงโครงสร้างและขั้นตอนการประมวลผลไฟล์ JSP

จากขั้นตอนการประมวลผลไฟล์ JSP ข้างต้น สามารถแบ่งออกได้เป็น 2 ช่วงหลักๆ คือ

- 1 . ช่วง translation ได้แก่ขั้นตอนข้อ 4 และข้อ 5 ซึ่งเป็นการแปลงเอกสาร JSP (ไฟล์ .jsp) ให้เป็น Servlet (ไฟล์ .java) จากนั้นก็จะคอมไพล์ไฟล์ Servlet ให้เป็นไฟล์ .class
- 2 . ช่วง execution ได้แก่ขั้นตอนข้อ 6 ซึ่งเป็นการนำเอาไฟล์ .class ที่ได้จากการคอมไพล์ มาประมวลผลหรือทำงานตามคำร้องขอจากไคลเอนต์นั่นเอง

3.2 เปรียบเทียบ JSP กับเทคโนโลยีอื่นๆ

3.2.1 JSP กับ Servlet

Servlet เป็นเทคโนโลยีที่เริ่มมาก่อน JSP และใช้ภาษาจาวาเป็นพื้นฐาน การทำงานของทั้งสองก็คล้ายกัน แต่ JSP จะมีขั้นตอนที่เพิ่มขึ้นมา คือ จะต้องแปลงเอกสาร JSP ให้เป็น Servlet ก่อน ผลลัพธ์สุดท้ายที่ได้ก็เป็น Servlet เหมือนกัน

Servlet จะยุ่งยากกว่าตรงที่ไม่สามารถแทรกแท็ก HTML เข้าไปได้ ต้องพิมพ์เข้าไปเอง โดยคำสั่ง `out.print()` แต่ JSP สามารถนำแท็ก HTML รวมกับแท็ก JSP ได้เลย ซึ่งจะสะดวกในการใช้งานมากกว่า Servlet

3.2.2 JSP กับ ASP

ความสามารถโดยรวมของ JSP จะคล้ายกับ ASP มาก เพราะสามารถแทรกแท็กเข้าไปในเอกสาร HTML ได้เหมือนกัน ต่างกันที่ JSP พัฒนามาบนพื้นฐานของภาษาจาวา มีข้อดีเหนือภาษา Visual Basic คือ สามารถจัดการข้อผิดพลาดและการทำงานในลักษณะ MultiThreading และ JSP สามารถทำงานได้ทุกระบบปฏิบัติการ

3.2.3 JSP กับ JavaScript

JavaScript เป็นการทำเนื้อหาไดนามิกให้กับเว็บเหมือนกัน แต่ JavaScript จะเกิดการประมวลผลที่ไคลเอนต์ ถ้าเว็บเบราว์เซอร์ไม่สนับสนุนจาวาสคริปต์ก็จะทำให้ไม่สามารถทำงานได้ แต่ JSP จะเกิดการประมวลผลที่เว็บเซิร์ฟเวอร์โดยตรงทำให้ไม่มีปัญหากับเว็บเบราว์เซอร์

3.3 แท็กพื้นฐานและคำสั่งควบคุม

ในการเขียนสคริปต์ก็คือ แทรกสคริปต์ลงไปในเอกสาร HTML โดยเรียกแท็กของ JSP ว่า scripting element หรือแท็กที่ใช้ในการเขียนสคริปต์ JSP แบ่งออกเป็น 3 ประเภท ได้แก่ scriptlet , expression และ declaration โดยที่แต่ละประเภทมีรูปแบบการใช้งานที่แตกต่างกัน

1 . scriptlet เป็นแท็กที่ใช้สำหรับเขียนคำสั่งภาษา Java ทั่วไป ซึ่งมีรูปแบบการเขียนดังนี้

```
<%      statement;      %>
```

สเตตเมนต์ (statement) คือประโยคคำสั่งที่เขียนขึ้นมาเพื่อทำงานอย่างใดอย่างหนึ่ง เช่น การคำนวณ , การใช้คำสั่งเงื่อนไข , การเรียกใช้เมธอดต่างๆ เป็นต้น โดยจะต้องมีเครื่องหมาย ; ปิดท้ายสเตตเมนต์แต่ละประโยคเสมอ

2 . expression เป็นแท็กที่ใช้สำหรับแสดงค่าของตัวแปรหรือเมธอดในรูปแบบย่อ

```
<%=      expression      %>
```

เอ็กซ์เพรสชัน (expression) อาจจะเป็นตัวแปรใดๆหรือการคำนวณตัวแปรหลายตัว หรืออาจจะเป็นเมธอดก็ได้ สังเกตว่าการระบุเอ็กซ์เพรสชัน ไม่จำเป็นต้องปิดท้ายด้วยเครื่องหมาย ;

3 . declaration เป็นแท็กที่ใช้ในการประกาศตัวแปรหรือเมธอดต่างๆ ที่จะนำมาใช้ในงานสคริปต์ จะต้องมีการประกาศขึ้นมาก่อน

```
<%!      declaration      %
```

ดีคลาเรชัน (declaration) อาจเป็นประโยชน์ของการประกาศตัวแปรหรือประกาศเมธอดก็ได้ ถ้าเป็นการประกาศตัวแปร จะต้องปิดท้ายด้วยเครื่องหมาย ; เสมอ ส่วนการประกาศเมธอดไม่ต้องปิดท้ายด้วยเครื่องหมาย ;

แท็กทั้ง 3 แบบที่ได้อธิบายไปข้างต้น สามารถประกาศไว้ที่ตำแหน่งใดก็ได้ภายในเอกสาร HTML อาจประกาศไว้ก่อนแท็ก <html> หรือประกาศภายในแท็ก <html> ก็ได้และในหมู่แท็ก scriptlet , expression , declaration ด้วยกันเอง เราจะระบุแท็กแบบไหนก่อน-หลังก็ได้ ในเอกสาร HTML

3.4 แท็กหมายเหตุ

ในการเขียนข้อความหมายเหตุหรือคอมเมนต์ (comment) ลงในไฟล์เอกสาร JSP สามารถเขียนได้ 3 รูปแบบ

1 . เขียนคอมเมนต์ในรูปแท็กที่ JSP กำหนดมาให้แต่ห้ามเขียนแทรกไว้ในแท็ก scriptlet , expression และ declaration ต้องเขียนไว้ลอยๆ ดังนี้

```
< % -      comment      - % >
```

2 . เขียนคอมเมนต์ตามรูปแบบของภาษา Java ซึ่งเขียนได้ทั้งใน scriptlet , expression และ declaration เช่นถ้าเขียนคอมเมนต์ในแท็ก scriptlet จะมีลักษณะดังนี้

```
<%
//      comment 1 line
/*      comment many line
      comment many line      */
%>
```

3 . เขียนคอมเมนต์ในรูปแบบของเอกสาร HTML หรือ XML แต่ห้ามเขียนแทรกไว้ใน scriptlet , expression และ declaration

```
<!--      comment in html tag      - >
```

3.5 Directive กับ Action

เป็นแท็กที่ใช้แทรกเข้าไปในเอกสาร HTML ทำให้กลายเป็นเอกสาร JSP เหมือนกับ scripting element เพียงแต่จุดประสงค์ในการใช้งานจะแตกต่างกันออกไป คือ เป็นแท็กที่ใช้กำหนดคุณสมบัติของไฟล์ JSP ที่สร้างขึ้นมา รวมทั้งใช้ในการติดต่อเชื่อมโยงกับออบเจกต์ต่างๆภายนอกด้วย เช่น การเรียกใช้ไฟล์ภายนอก , การสร้างแท็กตัวเอง , การ forward จากสคริปต์เดิมไปยังสคริปต์อื่นๆ

3.5.1 Directive

Directive มาตรฐาน มีอยู่ 3 ประเภท คือ page directive , include directive และ taglib directive ซึ่งมีการเขียนเหมือนกัน คือ เริ่มต้นด้วย `<%@` และปิดท้ายด้วย `%>`

1. page directive ใช้สำหรับกำหนดคุณสมบัติ (attribute) ต่าง ๆ ของเอกสาร JSP แอตทริบิวต์ที่กำหนดได้มี 12 รูปแบบ อาทิ info , language , import , contentType เป็นต้น ไฟล์ของเอกสาร JSP แต่ละไฟล์สามารถกำหนดแอตทริบิวต์ได้เพียงครั้งเดียวเท่านั้น ยกเว้น import สามารถกำหนดได้หลายๆครั้ง ตัวอย่างเช่น

```
<%@ language = "javascript" %>
<%@ import = "java.util.Random" %>
```

2 . include directive ไฟล์ JSP ที่มีการเรียกใช้ include directive หมายความว่า ให้นำเอาเนื้อความจากไฟล์อื่นๆที่ระบุไว้ใน include directive มาแทรกไว้ในไฟล์ JSP นั้น ณ ตำแหน่งที่วาง include directive โดยไฟล์ที่ถูก include เข้ามา อาจจะเป็นไฟล์ HTML หรือไฟล์ JSP เหมือนกันก็ได้ ตัวอย่างเช่น

```
<%@ include file = "header.jsp" %>
```

หมายความว่า header.jsp เป็นไฟล์ที่ถูก include เข้ามา เราสามารถวาง include directive ไว้ตำแหน่งใดในไฟล์ JSP ก็ได้

3 . taglib directive เป็นการบอก JSP Container ให้รู้ว่า ภายในสคริปต์ JSP จะมีการใช้แท็กที่สร้างขึ้นใหม่

3.5.2 Action

Action เป็นแท็กที่ใช้สำหรับควบคุมการส่งข้อมูลระหว่างเพจแต่ละเพจ รวมถึงการติดต่อกับออบเจกต์อื่นๆด้วย เช่น Java Applet หรือ คอมโพเนนท์ JavaBean เป็นต้น แท็ก action แบ่งออกเป็น 4 ประเภท คือ forward action , include action , plugin action และ JavaBean action ตัวอย่างเช่น

1 . forward action ใช้สำหรับส่งคำร้องขอจากไฟล์ JSP ไปยังไฟล์อื่นๆ ตามที่ระบุไว้

```
<jsp: forward page = "destination file" />
```

2 . include action ใช้แก้ปัญหาของ include directive ที่ไม่สามารถล่วงรู้การเปลี่ยนแปลงแก้ไขไฟล์ที่ include เข้ามา

```
<jsp: include page = "header.jsp" flush = "false" >
```

3 . plugin action ใช้สำหรับแทรกคอมโพเนนต์ (component) อื่นๆเข้ามาในไฟล์ JSP

```
<jsp : plugin type = "applet" code = "myapplet.class" width = "475" height = "350">
</jsp : plugin>
```

4 . JavaBean action ใช้เมื่อต้องการนำเอา JavaBean มาใช้ร่วมกับไฟล์ JSP โดย action มาตรฐาน อยู่ 3 แบบ คือ <jsp : useBean> , <jsp : setProperty> และ <jsp : getProperty>

3.6 Implicit Object

เป็นอ็อบเจกต์ที่ใช้งานบ่อยๆ ดังนั้น JSP Container จึงช่วยอำนวยความสะดวกโดยการสร้างอ็อบเจกต์เหล่านั้นขึ้นมาโดยอัตโนมัติ ทำให้เราสามารถเรียกใช้งานได้ทันทีโดยไม่ต้องประกาศก่อนใช้งาน มีอยู่ทั้งหมด 9 อ็อบเจกต์

อ็อบเจกต์	สร้างขึ้นมาจากคลาส...
request	javax.servlet.http.HttpServletRequest
response	javax.servlet.http.HttpServletResponse
session	javax.servlet.http.HttpSession
application	javax.servlet.ServletContext
out	javax.servlet.jsp.JspWriter
exception	java.lang.Throwable
pageContext	javax.servlet.jsp.PageContext
config	javax.servlet.ServletConfig
page	java.lang.Object

ตารางที่ 3-1 แสดง Implicit Object